



SH79F326 例程说明

1. 概述

1.1 整体框架介绍

本次提供以下模块的 Demo 程序：

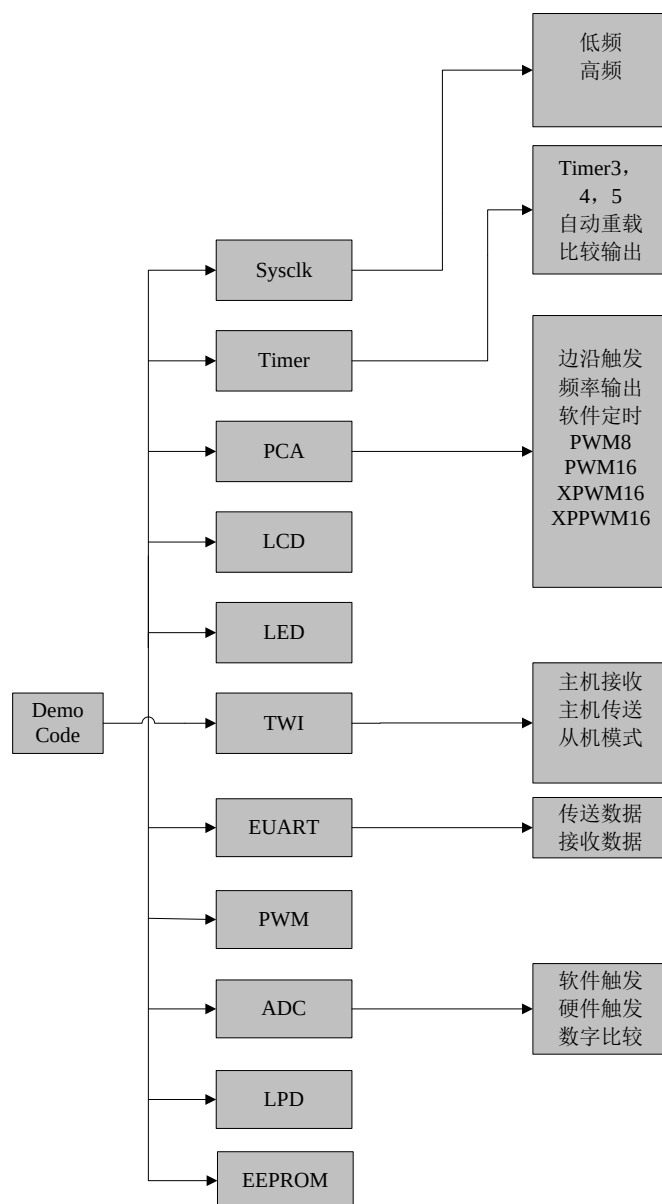
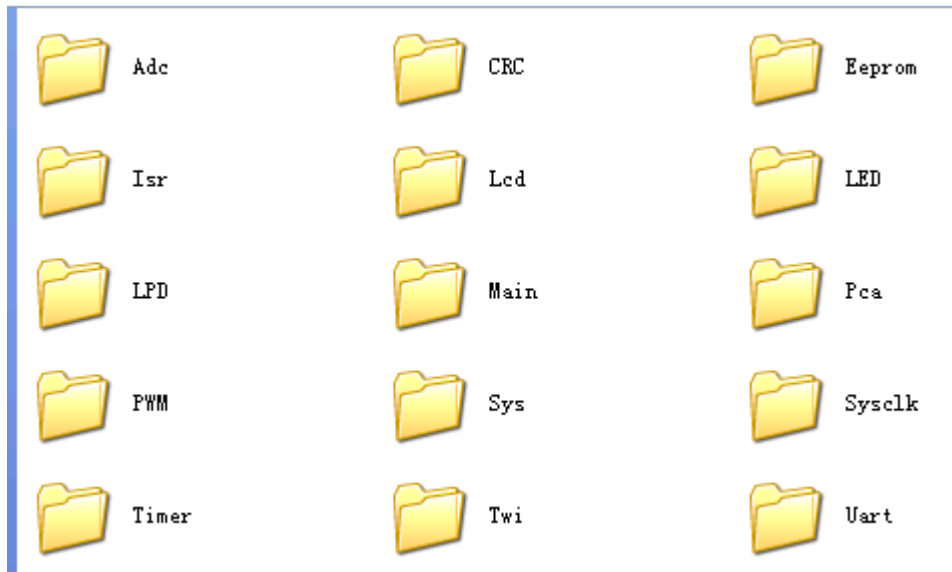


图 1-1

模块实例源程序如下：



每个模块包含两个.c 文件，一个命名为 xxx.c，一个命名为 xxx_Test.c；第一个 c 文件包含了模块相关的配置程序，第二个 c 文件包含了模块相关的测试程序；

1.2 使用实例说明

打开\Demo_code_SH79F326\Keil\326 Demo Program.uvproj，在 Keil 项目中选择包含相应模块程序的工程进行编译加载，对于有些模块，因为具有多个工作方式，为了测试不同工作方式下的功能，可在相应模块的.h 文件中打开相应的宏定义，再进行编译加载；例如，需要测试 PCA 工作在 MODE0 的功能，可按如下步骤进行测试：

- 1) 在工程下拉菜单中选择 PCA 工程（如图 1-2 所示）；

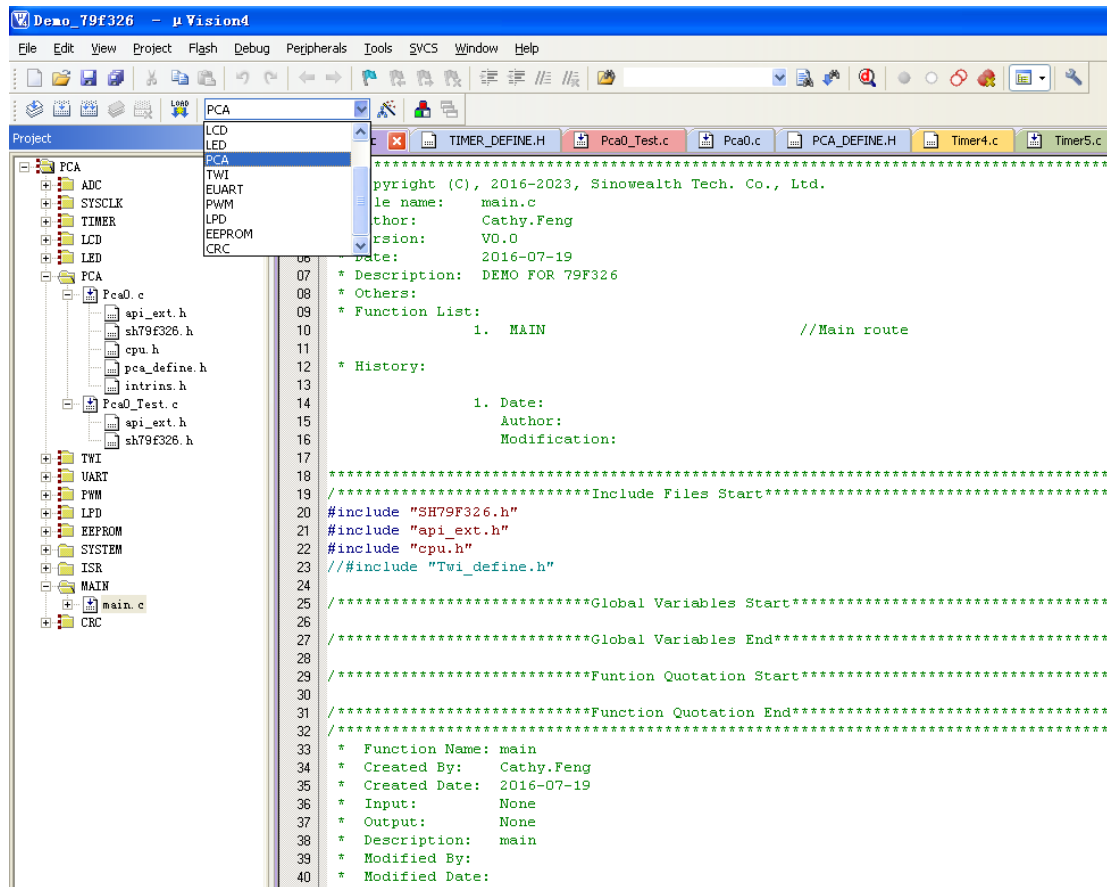


图 1-2

选择之后，包含 PCA 模块程序的工程被打开，如下图所示：

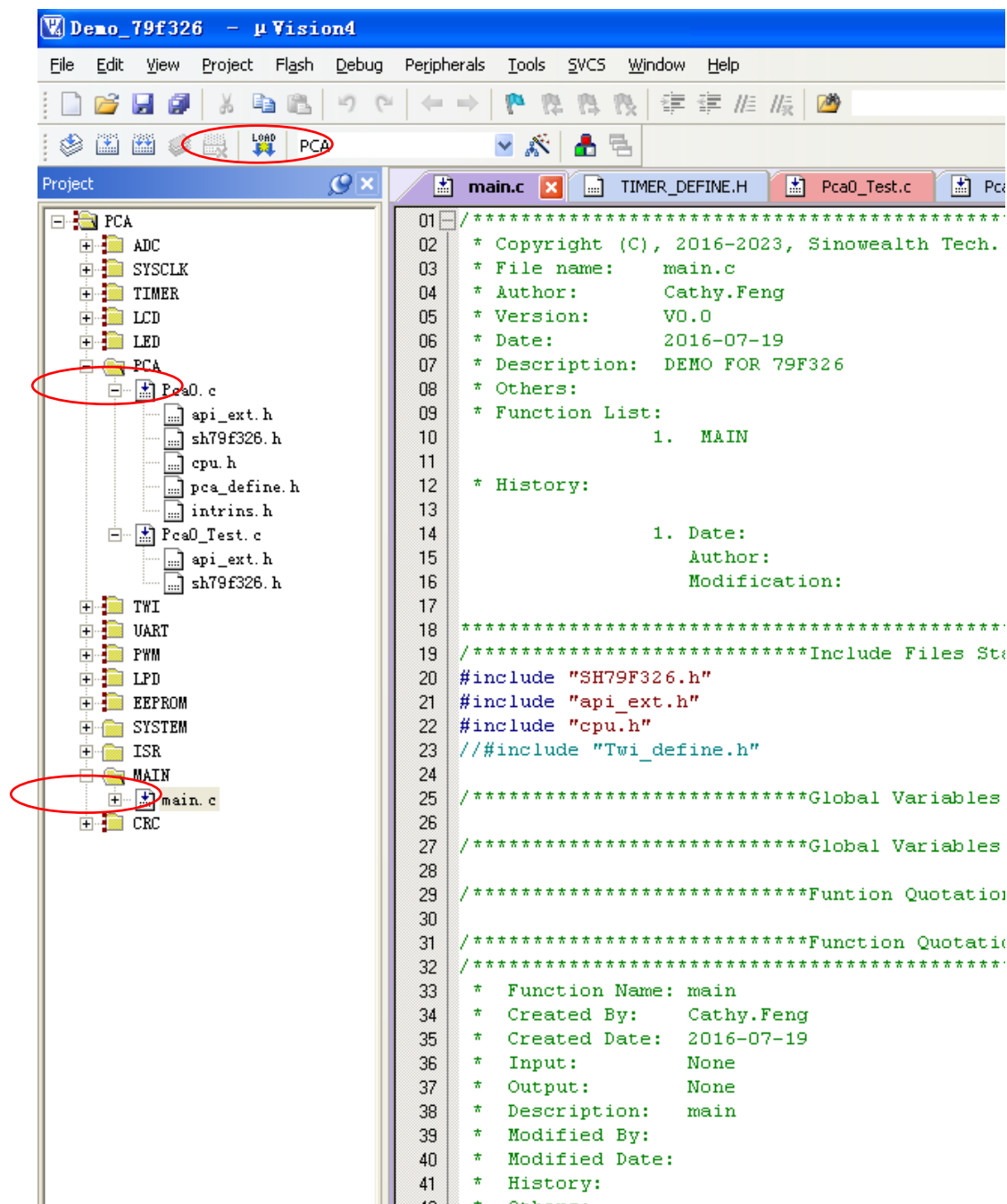


图 1-3

.c 文件中带“↓”，表示该文件包含在当前工程中，否则不包含在当前工程中。

2) 在相应的"pca_define.h"头文件中打开宏定义“#define MODE0”（如图 1-4 所示）；

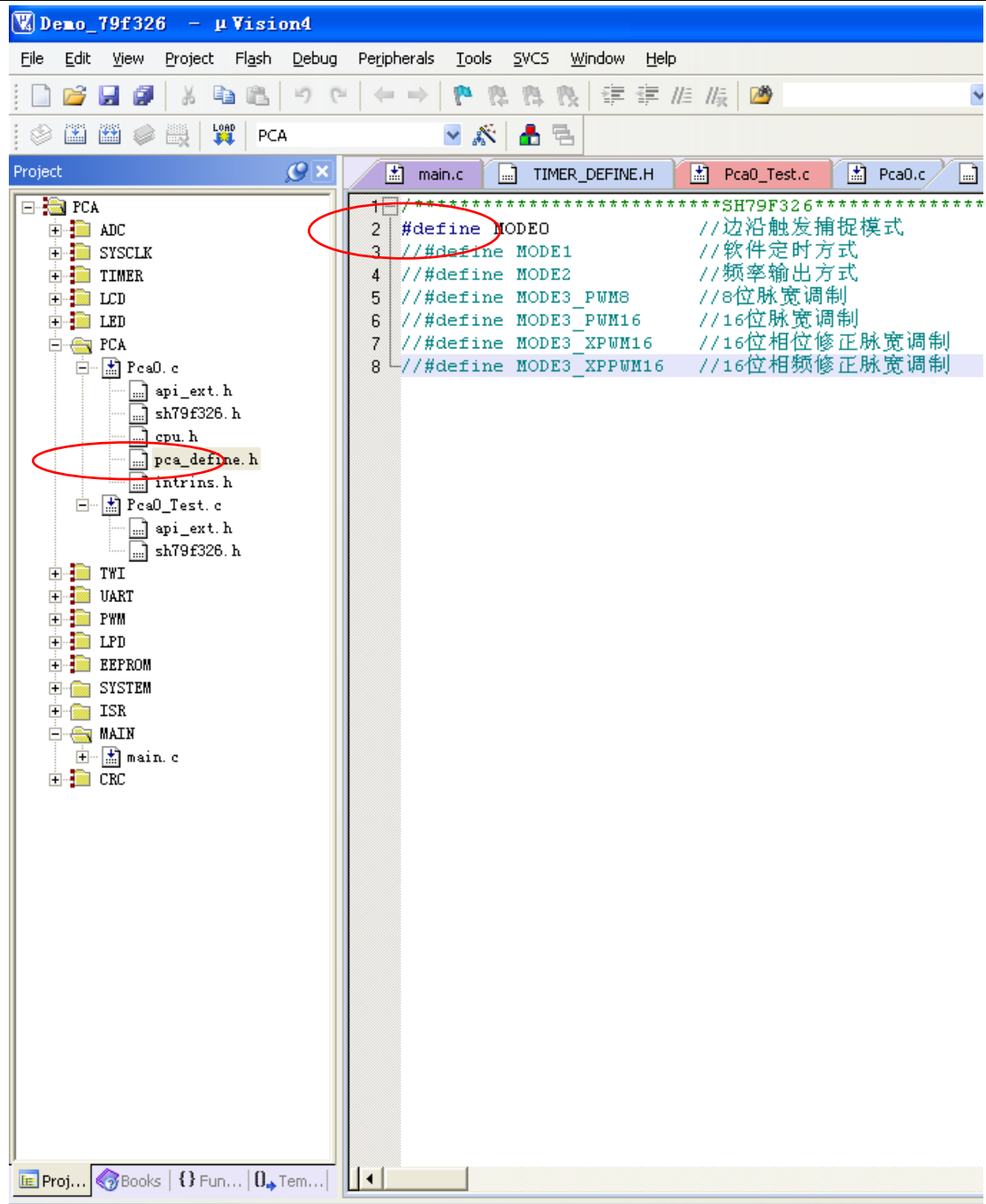


图 1-4

3) 编译、加载运行



2. 模块介绍

2.1. 系统时钟

2.1.1 工作方式配置介绍

系统时钟可配置为低频、高频 2 种模式，sysclk_define.h 文件包含以下几个宏定义，分别用于初始化时钟在相应的模式：

#define LOW_FREQUENCY：执行函数 void SetClk() 可将系统时钟配置为低频模式； **#define**

HIGH_FREQUENCY：执行函数 void SetClk() 可将系统时钟配置为高频模式；

将相应功能的宏定义打开，屏蔽其他宏定义，编译、加载运行可进行测试。

2.1.2 测试程序介绍

选择“Sysclk”工程，打开宏定义“**#define LOW_FREQUENCY**”，其它宏定义都屏蔽，设置系统时钟为低频时钟，Option 中选择内部 128kHz 或者 32.768 crystal；编译、加载运行，测试 P0.7 口可看到 IO 按照低频时钟频率翻转。

如果在双时钟系统中切换到高频时钟，打开宏定义“**#define HIGH_FREQUENCY**”，其它宏定义屏蔽，编译、加载运行，测试 P0.7 口可看到 IO 按照高频时钟频率翻转。

图由于 c 程序中 IO 翻转一次需要 7 个时钟周期，所以 IO 翻转产生的方波频率应为：

$$f = F_{\text{sys}} / (7 * 2), F_{\text{sys}} \text{ 为当前选中的低频或者高频时钟。}$$

注：如果不能观察到上述结果，则检查 option 是否配置正确；

2.1.3 寄存器简要介绍

CLKCON

位编号	位符号	说明
7	32K_SPDUP	32.768KHz 振荡器加速模式控制位 0: 32.768kHz 振荡器常规模式, 由软件清除 1: 32.768kHz 加速模式, 由软件或者硬件置起 详细说明见 Data Sheet
6-5	CLKS[1:0]	系统时钟预分频器 00: $f_{\text{SYS}} = f_{\text{OSC}}$ 01: $f_{\text{SYS}} = f_{\text{OSC}} / 2$ 10: $f_{\text{SYS}} = f_{\text{OSC}} / 4$ 11: $f_{\text{SYS}} = f_{\text{OSC}} / 12$ 如果选择 32.768kHz 为 SYSCLK 时钟源, 则 $f_{\text{SYS}} = f_{\text{OSC1}}$, 与 CLKS[1:0] 内容无关。



SH79F326 例程说明

3	HFON	OSCCLK 开关控制寄存器 0: 关闭 OSCCLK 1: 打开 OSCCLK
2	FS	频率选择位 0: 选择 OSC1CLK 为 OSCCLK 1: 选择 OSCCLK 为 OSCCLK



2.2 I/O 端口

SH79F326 提供 30 个位可编程双向 I/O 端口，端口数据在寄存器 P_x 中，端口控制寄存器 P_xCR_y 控制端口是作为输入或者输出。当端口作为输入时，每个 I/O 端口带有由 P_xPCR_y 控制的内部上拉电阻。

demo 程序中函数 `void Sysclk_Test()`，以 IO 翻转为例给出了 IO 的配置使用方法；



2.3 定时器/计数器

SH79F326 有定时器 3，定时器 4 和定时器 5 三个定时器。定时器 3 可配置为 16 位自动重载定时器；定时器 4 可配置为 16 位自动重载定时器；定时器 5 可配置为 16 位的自动重载定时器。

demo 程序给出了不同三个定时器的配置方法。

2.3.1 工作方式配置介绍

timer_define.h 文件中包含以下几个宏：

#define TIMER3: 设置 Timer3 工作在 32.768kHz 时钟源下的自动重载模式。

#define TIMER4: 设置 Timer4 工作在系统时钟的自动重载模式。

#define TIMER5: 设置 Timer5 工作在系统时钟的自动重载模式。

将相应功能的宏定义打开，屏蔽其他宏定义，编译、加载运行可进行测试。

2.3.2 测试程序介绍

选择“Timer”工程，依次在 timer_define.h 文件中打开相应宏，然后进行编译加载全速运行，观察现象

1. 当打开宏“#define TIMER3”，全速运行后，系统时钟 Option 选择 32.768kHz，P0_7 引脚输出方波。
2. 当打开宏“#define TIMER4”，全速运行后，时钟为系统时钟，P0_7 引脚输出方波。
3. 当打开宏“#define TIMER5”，全速运行后，时钟为系统时钟，P0_7 引脚输出方波。

输出时钟频率计算公式：

$$\text{Clock Out Frequency} = \frac{1}{2} \times \frac{f_{\text{SYS}}}{65536 - [\text{THX}, \text{TLX}]}$$

2.3.3 相关寄存器介绍

TIMER3

位编号	位符号	说明
7	TF3	定时器 3 溢出标志位 0: 无溢出（硬件清 0） 1: 溢出（硬件置 1）
5-4	T3PS[1:0]	定时器 3 预分频比选择位 00: 1/1 01: 1/8 10: 1/64 11: 1/256



SH79F326 例程说明

2	TR3	定时器 3 允许控制位 0: 停止定时器 3 1: 开始定时器 3
1-0	T3CLKS[1:0]	定时器 3 定时器/计数器方式选定位 00: 系统时钟, T3 引脚用作 I/O 端口 10: 外部 32.768kHz 晶体谐振器或 128k RC 其它: 保留

TIMER4

位编号	位符号	说明
7	TF4	定时器 4 溢出标志位 0: 无溢出 (硬件清 0) 1: 溢出 (硬件置 1)
5-4	T4PS[1:0]	定时器 4 预分频比选择位 00: 1/1 01: 1/8 10: 1/64 11: 1/256
1	TR4	定时器 4 允许控制位 0: 禁止定时器 4 1: 允许定时器 4

TIMER5

位编号	位符号	说明
7	TF5	定时器 5 溢出标志位 0: 无溢出 (硬件清 0) 1: 溢出 (硬件置 1)
5-4	T5PS[1:0]	定时器 5 预分频比选择位 00: 1/1 01: 1/8 10: 1/64 11: 1/256
1	TR5	定时器 5 允许控制位 0: 禁止定时器 5 1: 允许定时器 5



2.4 可编程计数器阵列 PCA0

demo 程序以 PCA0 为例，给出了其比较/捕捉模块分别工作于 Mode0~Mode3 时的参考程序。

2.4.1 工作方式配置介绍

pca_define.h 文件中包含以下几个宏：

#define MODE0: 配置比较/捕捉模块 0 工作在边沿触发模式

#define MODE1: 配置比较/捕捉模块 0 工作在软件定时方式

#define MODE2: 配置比较/捕捉模块 0 工作在频率输出方式

#define MODE3_PWM8: 配置比较/捕捉模块 0 工作在 8 位脉宽调制

#define MODE3_PWM16: 配置比较/捕捉模块 0 工作在 16 位脉宽调制

#define MODE3_XPWM16: 配置比较/捕捉模块 0 工作在 16 位相位修正脉宽调制

#define MODE3_XPPWM16: 配置比较/捕捉模块 0 工作在 16 位相频修正脉宽调制

将相应功能的宏定义打开，屏蔽其他宏定义，编译、加载运行可进行测试。

2.4.2 测试程序介绍

选择“PCA”工程，依次在 pca_define.h 文件中打开相应宏，然后进行编译加载全速运行，观察现象：

1. 打开宏“**#define MODE0**”，屏蔽其他宏定义，编译加载后，全速运行，P0CEX0 引脚上升沿/下降沿会触发一次捕捉，并产生相应中断；
2. 打开宏“**#define MODE1**”，屏蔽其他宏定义，编译加载后，全速运行，可以实现连续软件定时，当 PCA0 计数值与 P0CPH0 和 P0CPL0 中的值匹配时产生一次中断，同时 P0CEX0 口翻转一次；(P0TOP,P0CPn 可变)
3. 打开宏“**#define MODE2**”，屏蔽其他宏定义，编译加载后，全速运行，P0CEX0 引脚可输出如下波形（3.965KHz）：

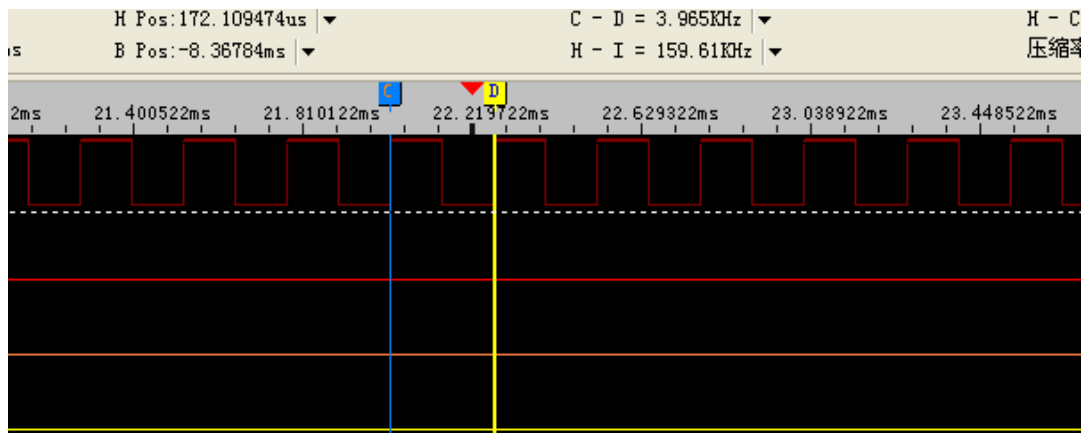


图 2-4-1

$$FP0CEXn = FPCA0 / (2 \times P0CPHn)$$

如果应用中需要改变波形占空比和频率,则可通过适时地改变 P0CPHn 来获得相应的波形;P0TOPL 固定为 0xFF,用户可以配置 P0TOPH 来改变计数最大值;

4. 打开宏“#define MODE3_PWM8”,屏蔽其他宏定义,编译加载后,全速运行,P0CEX0 输出占空比为 0.8 的正向波形,P0CEX1 输出占空比为 0.8 的反向波形,如下图所示:

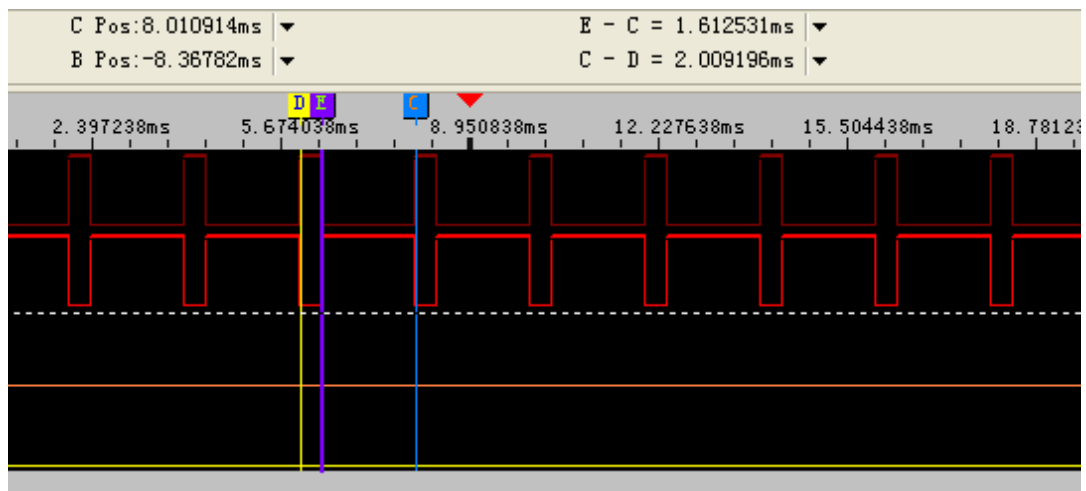


图 2-4-2

$$Duty = (256 - (P0CPHn + 1)) / 256$$

如果应用中需要 Duty 可变的 PWM 波形,则可通过适时的改变 P0CPHn 来获取希望波形;

P0TOPL 不可改变;

5. 打开宏“#define MODE3_PWM16”,屏蔽其他宏定义,编译加载后,全速运行,P0CEX0 输出占空比为 0.8 的正向波形,P0CEX1 输出占空比为 0.8 的反向波形,如下图所示:

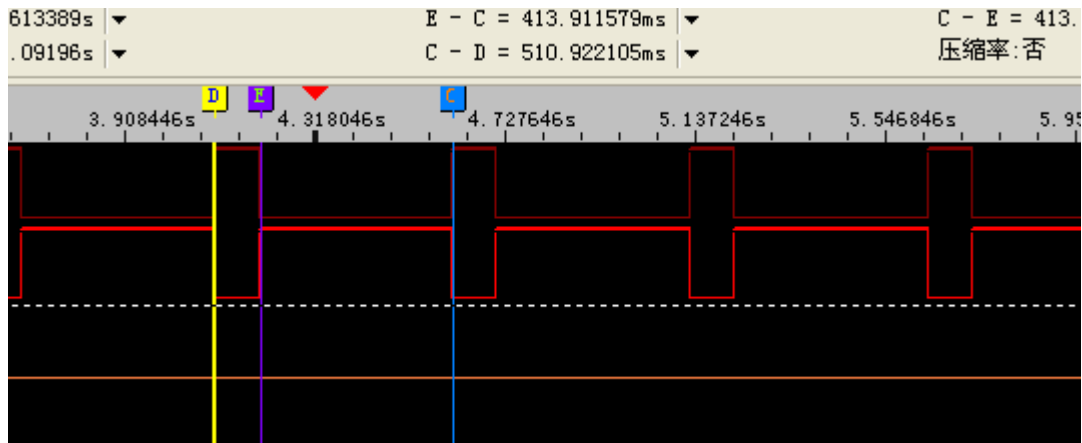


图 2-4-3

$$\text{Duty} = (65536 - (\text{P0CPn} + 1)) / 65536$$

如果应用中需要 Duty 可变的 PWM 波形，则可通过适时的改变 P0CPn (P0CPHn, P0CPLn) 和 P0TOP 来获取希望波形；

6. 打开宏“#define MODE3_XPWM16”，屏蔽其他宏定义，编译加载后，全速运行，P0CEX0 输出频率为 250Hz 的正向波形，P0CEX1 输出频率为 250Hz 的反向波形，如下图所示：

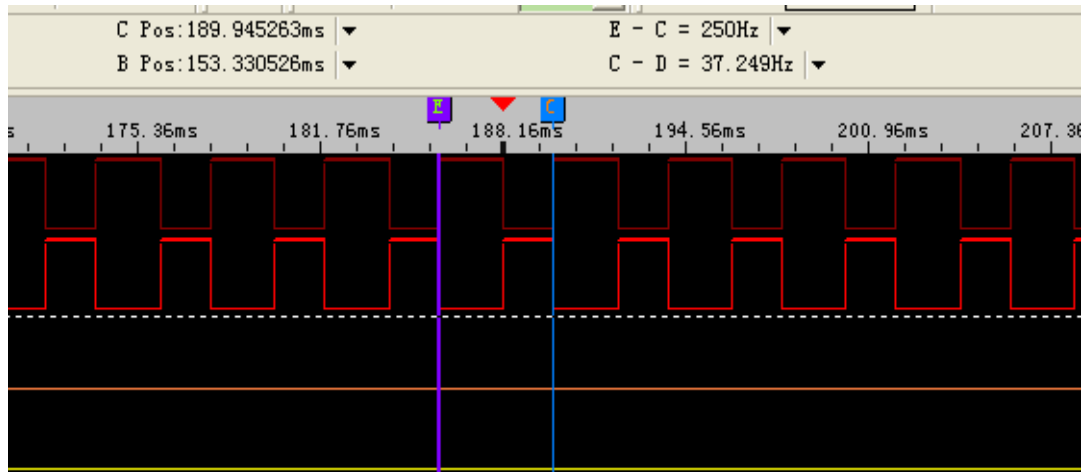


图 2-4-4

如果应用中需要频率和占空比可变的波形，则可通过适时的改变 P0TOP 和 P0CPn (P0CPHn, P0CPLn) 来获取相应波形；在一个定时器时钟周期里 PCA0 值等于 P0TOP 值，然后在下一次计数到来时 P0TOP 和 P0CPn 将得到更新。

7. 打开宏“#define MODE3_XPPWM16”，屏蔽其他宏定义，编译加载后，全速运行，P0CEX0 输出频率为 250Hz 的正向波形，P0CEX1 输出频率为 250Hz 的反向波形，如下图所示：

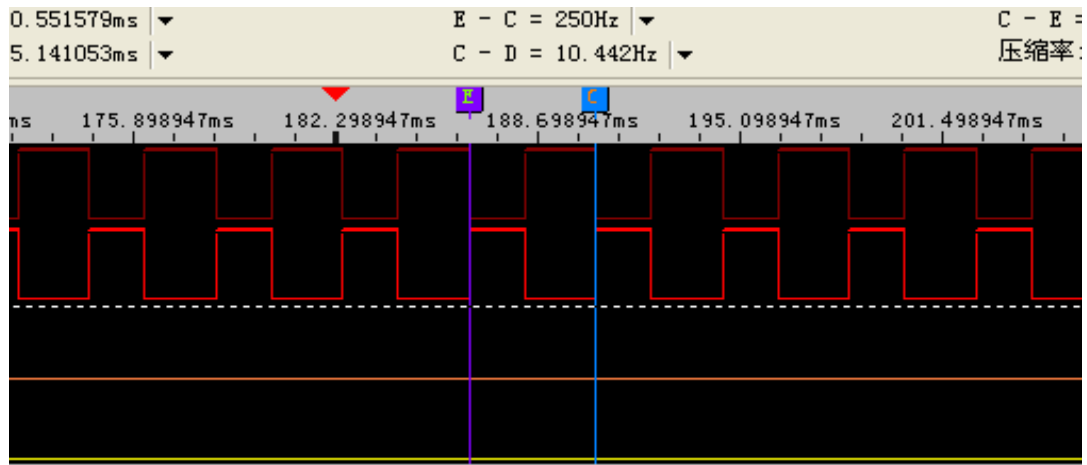


图 2-4-5

如果应用中需要频率和占空比可变的波形，则可通过适时的改变 P0TOP 和 P0CPn(P0CPHn,P0CPLn) 来获取相应波形；P0CPn 和 P0TOP 在 0x0000 点更新。

注：对于 XPWM 和 XPPWM 工作方式，一定要配置 PCA0 工作在双沿模式，否则相应口将不能正常输出波形；



2.5 LCD 驱动器

Demo 程序给出了 LCD 的使用实例，以 4COM，16SEG，帧频 64Hz 为例，用户可根据实际需要选择合适 COM 数和 SEG 数做相应的 IO 配置；配置参考 void init_lcd()。

用户可通过配置 DISPCON1 中 DUTY[2:0]来适应不同的 COM 应用:000 为 4COM×16SEG（其中，PIN 图中的 COM5~COM8 共享为 SEG25~SEG28）；001 为 8COM×12SEG；010 为 4COM×16SEG（其中 COM1~COM4 共享为 SEG25~SEG28）；011 为 5COM×15SEG（其中 COM6~COM8 共享为 SEG25~SEG27）；100 为 6COM×14SEG（COM7~COM8 共享为 SEG25~SEG26,1/3 偏置）；101：为 6COM×14SEG（COM7~COM8 共享为 SEG25~SEG26,1/4 偏置）。

选择“Lcd”工程，编译加载运行，接 4COM，16SEG 的屏幕，为全显；用示波器观察 SEG1~8，13~16，25~28 为选中波形、COM1~COM4 为 COM 口波形。

2.5.1 寄存器简要介绍

位编号	位符号	说明
7	LCDON	LCD 使能控制位 0: 禁止 LCD 驱动器 1: 允许 LCD 驱动器
6	MODSW	LCD 共享总选择位 0: LCD 端口控制位有效 1: 所有 LCD 共享端口设置为 IO 注意: 代 码选项 OP_MODSW=1，则当 MODSW = 1，当前 LCD 扫描数据将会被保留，LCD 引脚将会变成 I/O 引脚，当 MODSW = 0/LCD 引脚将会变回功能引脚，并接着由保留数据继续扫描。 代码选项 OP_MODSW = 0，则当 MODSW = 1，当前 LCD 扫描数据将不会被保留，LCD 模块时钟由内部时钟继续运行，LCD 引脚将会变成 I/O 引脚，当 MODSW = 0，LCD 引脚将会变回功能引脚，并接着由 LCD 模块内部时钟运行的时序切回继续扫描。
5	ELCC	LCD 对比度控制使能位 0: 关闭 LCD 对比度控制 1: 打开 LCD 对比度控制



SH79F326 例程说明

3-0	VOL[3:0]	LCD 对比度控制位 0000: $V_{LCD} = 0.531V_{DD}$ 0001: $V_{LCD} = 0.563V_{DD}$ 0010: $V_{LCD} = 0.594V_{DD}$ 0011: $V_{LCD} = 0.625V_{DD}$ 0100: $V_{LCD} = 0.656V_{DD}$ 0101: $V_{LCD} = 0.688V_{DD}$ 0110: $V_{LCD} = 0.719V_{DD}$ 0111: $V_{LCD} = 0.750V_{DD}$ 1000: $V_{LCD} = 0.781V_{DD}$ 1001: $V_{LCD} = 0.813V_{DD}$ 1010: $V_{LCD} = 0.844V_{DD}$ 1011: $V_{LCD} = 0.875V_{DD}$ 1100: $V_{LCD} = 0.906V_{DD}$ 1101: $V_{LCD} = 0.938V_{DD}$
-----	----------	---

位编号	位符号	说明
7-5	DUTY[2:0]	LCD 占空比选择位（与 DUTY0 组合控制） 000: 1/4 占空比, 1/3 偏置 (4 COM X 16 SEG) COM 口: COM1-4 SEG 口: SEG1-8,13-16,25-28 COM5-8 共享为 SEG25-28 001: 1/8 占空比, 1/4 偏置 (8 COM X 12 SEG) COM 口: COM1-8 SEG 口: SEG1-8,13-16 010: 1/4 占空比, 1/3 偏置 (4 COM X 16 SEG) COM 口: COM5-8 共享为 COM1-4 SEG 口: SEG1-8,13-16, COM1-4 共享为 SEG25-28 011: 1/5 占空比, 1/3 偏置 (5 COM X 15 SEG) COM 口: COM1-5 SEG 口: SEG1-8,13-16,25-27, COM6-8 共享为 SEG25-27 100: 1/6 占空比, 1/3 偏置 (6 COM X 14 SEG) COM 口: COM1-6 SEG 口: SEG1-8,13-16,25-26, COM7-8 共享为 SEG25-SEG26 101: 1/6 占空比, 1/4 偏置 (6 COM X 14 SEG) COM 口: COM1-6 SEG 口: SEG1-8,13-16,25-26, COM7-8 共享为 SEG25-SEG26 其它: 1/4 占空比, 1/3 偏置 (4 COM X 16 SEG) COM 口: COM1-4
4	RLCD	LCD 偏置电阻选择控制位 0: LCD 偏置电阻为 900k 1: LCD 偏置电阻总和为 1.5M
3-2	FCCTL[1:0]	充电时间控制位 00: 1/8 LCD com 周期 01: 1/16 LCD com 周期 10: 1/32 LCD com 周期 11: 1/64 LCD com 周期



SH79F326 例程说明

1-0	MOD[1:0]	驱动模式选择位 00: 传统电阻型模式，偏置电阻总和为 900k/1.5M 01: 传统电阻型模式，偏置电阻总和为 60k 10: 快速充电模式，偏置电阻总和自动在 60k 和 900k/1.5M 之间切换 11: 无意义
-----	----------	--

其他相关寄存器详见 Data Sheet.



2.6 LED 驱动器

Demo 程序给出了 2 种模式的 LED 的使用实例。模式一以 8COM, 12SEG 为例, 用户可根据实际需要选择合适 COM 数和 SEG 数做相应的 IO 配置; 模式二以呼吸灯为例, 配置参考 void init_led()。

模式一和模式二在 LED_DEFINE.H 中选择。

选择“Led”工程, 编译加载运行, 观察 COM1~COM8、SEG1~12SEG 口波形;

2.6.1 寄存器简要介绍

位编号	位符号	说明
7	LEDON	LED 使能控制位 0: 禁止 LED 驱动器模块 1: 允许 LED 驱动器模块
6	LEDMD	LED 中断运行模式控制位 0: LEDIF=1 时, LED 扫描继续。 1: LEDIF=1 时, LED 扫描停止, 需要将 LEDON 置 1 开始下一帧扫描。
5	MODE	LED 模式选择位 0: 模式 1 (不可调光的 LED 显示) 1: 模式 2 (带可调光的 LED 显示)
4	LEDIF	LED_FRAME 中断标志 0: 无 LED 帧中断, 由软件清 0 1: 由硬件置 1, 表示已完成一帧的 LED 扫描。
3	COMIF	LED_COM 中断标志 0: 无 LED_COM 中断, 由软件清 0 1: 由硬件置 1, 表示已完成一个的 LED_COM 扫描。
0	MODSW	LED 共享总选择位 0: LED 端口控制位有效 1: 所有 LED 共享端口设置为 IO 注意: 代码选项 OP_MODSW=1, 则当 MODSW = 1, 当前 LED 扫描数据将会被保留, LED 引脚将会变成 I/O 引脚, 当 MODSW = 0, LED 引脚将会变回功能引脚, 并接着由保留数据继续扫描。 代码选项 OP_MODSW = 0, 则当 MODSW = 1, 当前 LED 扫描数据将不会被保留, LED 模块时钟由内部时钟继续运行, LED 引脚将会变成 I/O 引脚, 当 MODSW = 0, LED 引脚将会变回功能引脚, 并接着由 LED 模块内部时钟运行的时序切回继续扫描。

其它相关寄存器详见 Data Sheet。



2.7 TWI 串行通讯接口

TWI 串行总线采用两根线(SDA 和 SCL)在总线和装置之间传递信息。SH79F326 完全符合 TWI 总线规范，自动对字节进行传输进行处理，并对串行通讯进行跟踪。

2.7.1 工作方式配置介绍

twi_define.h 文件中定义了以下几个模式：

#define M_MODE_Send: 打开该宏，配置 SH79F326 工作在主机发送模式；

#define M_MODE_Receive: 打开该宏，配置 SH79F326 工作在主机接收模式；

#define S_MODE_Send: 打开该宏，配置 SH79F326 工作在从机发送模式；

#define S_MODE_Receive: 打开该宏，配置 SH79F326 工作在从机接收模式；

将相应功能的宏定义打开，屏蔽其他宏定义，编译、加载运行可进行测试。

2.7.2 测试程序介绍

Twi_Test.c 分别以作为主机和从机传送和接收 8 个 byte 数据为例给出了该模块的应用：

1. 选择“Twi”工程后，打开宏定义“M_MODE_Send”，编译、加载运行可测试 326 作为主机向从机传送数据，分别为“0x10,0x20,0x30,0x40,0x50,0x60,0x70,0x80”
2. 选择“Twi”工程后，打开宏定义“S_MODE_Receive”，编译、加载运行可测试 326 作为从机从主机接收 8 个 byte 数据；
3. 选择“Twi”工程后，打开宏定义“S_MODE_Send”，编译、加载运行可测试 326 作为从机向主机传送数据，分别为“0x10,0x20,0x30,0x40,0x50,0x60,0x70,0x80”
4. 选择“Twi”工程后，打开宏定义“M_MODE_Receive”，编译、加载运行可测试 326 作为主机从从机接收 8 个 byte 数据；

本模块测试需要两个 326 板子，一个作为主机，一个作为从机进行配合。

2.8 Uart 接口

Uart 工作方式列表：

SM0	SM1	方式	类型	波特率	帧长度	起始位	停止位	第 9 位
0	0	0	同步	$f_{SYS}/(4 \text{ 或 } 12)$	8 位	无	无	无
0	1	1	异步	自带波特率发生器的溢出率/16	10 位	1	1	无
1	0	2	异步	$f_{SYS}/(32 \text{ 或 } 64)$	11 位	1	1	0, 1
1	1	3	异步	自带波特率发生器的溢出率/16	11 位	1	1	0, 1

通过配置相应寄存器可使 Uart 工作在不同的工作方式，本次 Demo 程序以 Uart0 的方式 1 为例，给出



了 326 传送数据和接收数据的实例。

注：Uart0 和 Uart1 完全相同；

2.8.1 工作方式配置介绍

uart_define.h 文件中，定义了以下两个宏：

#define Enable_uart_TX_test: 测试 326 工作在方式 1 传送数据；

#define Enable_uart_RX_test: 测试 326 工作在方式 1 接收数据；

2.8.2 测试程序介绍

本项测试需结合串口助手调试，选择“Uart”工程后，打开宏定义“#define Enable_uart_TX_test”，编译、加载运行后，通过串口助手可看到 326 连续传送 0x96；

注：测试时，系统时钟选择外挂晶振 8M，波特率为 9600；

2.8.3 寄存器介绍

PCON

位编号	位符号	说明
7	SMOD	UART0/1 波特率加倍器 0: 在方式 2 中，波特率为系统时钟的 1/64 1: 在方式 2 中，波特率为系统时钟的 1/32
6	SSTAT	SCON[7:5]功能选择 0: SCON[7:5]工作方式作为 SM0, SM1, SM2 1: SCON[7:5]工作方式作为 FE, RXOV, TXCOL
3-2	GF[1:0]	用于软件的通用标志位
1	PD	掉电模式控制位
0	IDL	空闲模式控制位

SCON

位编号	位符号	说明
7-6	SM[0:1]	EUART0 串行方式控制位，SSTAT = 0 00: 方式 0，同步方式，固定波特率 01: 方式 1，8 位异步方式，可变波特率 10: 方式 2，9 位异步方式，固定波特率 11: 方式 3，9 位异步方式，可变波特率
7	FE	EUART0 帧出错标志位，当 FE 位被读时，SSTAT 位必须被置位 0: 无帧出错，由软件清零 1: 帧出错，由硬件置位
6	RXOV	EUART0 接收溢出标志位，当 RXOV 位被读时，SSTAT 位必须被置位 0: 无接收溢出，由软件清零 1: 接收溢出，由硬件置位



5	SM2	EUART0 多处理机通讯允许位（第 9 位“1”校验器），SSTAT = 0 0: 在方式 0 下，波特率是系统时钟的 1/12 在方式 1 下，禁止停止位确认检验，任何停止位都会置位 RI 在方式 2 和 3 下，任何字节都会置位 RI 1: 在方式 0 下，波特率是系统时钟的 1/4 在方式 1 下，允许停止位确认检验，只有有效的停止位（1）才能置位 RI 在方式 2 和 3 下，只有地址字节（第 9 位 = 1）才能置位 RI
5	TXCOL	EUART0 发送冲突标志位，当 TXCOL 位被读时，SSTAT 位必须被置位 0: 无发送冲突，由软件清零 1: 发送冲突，由硬件置位
4	REN	EUART0 接收器允许位 0: 接收禁止 1: 接收允许
3	TB8	在 EUART0 的方式 2 和 3 下发送的第 9 位，由软件置位或清零
2	RB8	在 EUART0 的方式 1, 2 和 3 下接收的第 9 位 在方式 0 下，不使用 RB8 在方式 1 下，如果接收中断发生，停止位移入 RB8 在方式 2 和 3 下，接收第 9 位
1	TI	EUART0 的传送中断标志位 0: 由软件清零 1: 由硬件置位
0	RI	EUART0 的接收中断标志位 0: 由软件清零 1: 由硬件置位

SBUF

位编号	位符号	说明
7-0	SBUF[7:0]	这个寄存器寻址两个寄存器：一个移位寄存器和一个接收锁存寄存器 SBUF 的写入将发送字节到移位寄存器中，然后开始传输 SBUF 的读取返回接收锁存器中的内容

SADDR、SADEN(多机通讯时需设置)

位编号	位符号	说明
7-0	SADDR[7:0]	寄存器 SADDR 定义了 EUART0 的从机地址
7-0	SADEN[7:0]	寄存器 SADEN 是一个位屏蔽寄存器，决定 SADDR 的哪些位被检验 0: SADDR 中的相应位被忽略 1: SADDR 中的相应位对照接收到的地址被检验

SBRTH、SBRTL

位编号	位符号	说明
7	SBRTEN	EUART0 波特率发生器使能控制位 0: 关闭（默认） 1: 打开
6-0, 7-0	SBRT[14:0]	EUART0 波特率发生器计数器高 7 位和低 8 位寄存器

SFINE

位编号	位符号	说明
3-0	SFINE[3:0]	EUART0 波特率发生器微调数据寄存器



2.9 ADC

ADC 负责模拟信号到相应 12 位的数字信号的转换，支持 VDD 和外接 VREF 两种基准电压，可通过软件启动 AD 转换，也可通过触发源触发 AD 转换，另外可配置为多通道，实现序列转换。

2.9.1 测试程序介绍

Demo 程序中配置了单次转换和 8 通道的序列转换。单次转换使用 AN0 为输入口。序列转换的顺序为 AN7~AN0,转换完成后，结果按照 AN7~AN0 的顺序存放在 ADC_res[8]数组中。

单次转换需要把 Adc.c 文件中的 define ADC_ONE 打开。

序列转换需要把 Adc.c 文件中的 define ADC_ARRAY 打开。

选择 “Adc” 工程，编译、加载

- 1) AN0~AN7 通道外接不同的电压源；
- 2) 在 ADC_Test 最后设置断点，可读出 ADC_res 中的转换结果。

注：本项测试只完成一次 AD 转换，要想再进行一次 AD 转换，软件需再次启动 AD 转换。

2.9.2 寄存器简要介绍

功能	名称	寄存器描述
ADC 时钟设置	ADT	设置 ADC 时钟与采样时间
ADC 控制	ADCON1	AD 模块使能、启动、参考电压的选择、及 ADC 转换完成中断标志、事件触发设置
	ADCON2	序列信道总数设置、相邻通道之间时间间隔设置
映像控制	SEQCON	通道及转换结果映像控制、转换结果对齐方式设置
AD 信道配置 1	ADCH1	设置 AD 信道引脚为 AD 信道功能或 I/O 功能
AD 信道配置 2	ADCH2	设置 AD 信道引脚为 AD 信道功能或 I/O 功能
通道和转换顺序设置	SEQCHx	指定序列中的通道以及转换顺序，x = 0 - 7
ADC 结果寄存器	ADDxL	SEQCHx 中指定通道转换值的低位，x = 0 - 7
	ADDxH	SEQCHx 中指定通道转换值的高位，x = 0 - 7

寄存器的每一位详细介绍见 Data Sheet。



2.10 PWM

SH79F326 含两路 12bit PWM，可以产生 2 路周期和占空比分别可以调整的脉宽调制波形。

Demo 程序中完成了 2 路互补输出的 PWM。

2.10.1 测试程序介绍

选择工程“PWM”，编译、加载运行，测量 P3.0 和 P2.6 的波形；

注：实际应用中，可根据需要调整 PWM 的时钟，周期以及占空比。

PWM 输出周期 = [PWMxPH, PWMxPL] X PWM 时钟周期。

2.10.2 寄存器介绍

PWMxCON

位编号	位符号	说明
7	PWMxEN	PWMx 使能位 0: 禁止 PWMx 模块 1: 允许 PWMx 模块
6	PWMxS	PWMx 输出模式 0: PWMx 占空比期间输出高电平，占空比溢出后输出低电平 1: PWMx 占空比期间输出低电平，占空比溢出后输出高电平
5-3	PWMxCK[2-0]	PWMx 时钟选择位 000: 系统时钟/1 001: 系统时钟/2 010: 系统时钟/4 011: 系统时钟/8 100: 系统时钟/16 101: 系统时钟/32 110: 系统时钟/64
2	PWMxIE	PWMx 中断使能位（当 IEN2 寄存器中的 EPWMx 位置 1） 0: 禁止 PWMx 周期中断 1: 允许 PWMx 周期中断
1	PWMxIF	PWMx 中断标志位 0: PWMx 周期计数器没有溢出 1: PWMx 周期计数器溢出，由硬件置 1
0	PWMxSS	PWMx 引脚输出控制位 0: PWMx 输出禁止，用作 I/O 等功能 <i>注：如果此位为 0 而 PWMxEN = 1，则整个 PWMx 模块仍然正常运行，只是波形输出被禁止，PWMx 模块可以做一个定时器来使用。</i> 1: PWMx 输出允许

PWMXPH/L

位编号	位符号	说明
11-0	PWMXP[11:0]	12 位 PWMX 周期寄存器

PWMXDH/L



SH79F326 例程说明

位编号	位符号	说明
3-0 7-0	PWMXP[11:0]	12 位 PWMX 周期寄存器



2.11 EEPROM 操作

本次 Demo 程序给出了读写 EEPROM 的接口函数，如下：

1) `UINT8 EEPromByteRead(UINT8 nAddrH,UINT8 nAddrL)`

从 EEPROM 指定地址处读取 1 个 byte；

2) `void EEPromByteProgram(UINT8 nAddrH,UINT8 nAddrL, UINT8 nData)`

向 EEPROM 指定地址处写一个 byte；

3) `void EEPromSectorErase(UINT8 nAddrH)`

擦除 EEPROM 指定扇区中内容；

注：在对 EEPROM 进行擦除和写操作之前，一定要关闭中断，待操作结束后，才可打开中断；另外，对 Flash 的读、写、擦除操作过程类似于类 EEPROM 操作，唯一区别是，在擦除或写、读之前应将 FAC 位清 0。

2.12 LPD

2.12.1 测试程序介绍

Demo 程序中设置当 Vdd 小于 4.65V 时，LPD 发生，P0.7 口不断翻转。当 Vdd 电压大于 4.65V 时，LPD 不发生，P0.7 停止翻转。

2.12.2 寄存器介绍

LPDCON

位编号	位符号	说明
7	LPDEN	LPD 允许位 0: 禁止低电压检测 1: 允许低电压检测
6	LPDF	LPD 标志位 0: 无 LPD 发生，由硬件清 0 1: LPD 发生，由硬件置 1，即当前电压高于/低于在 LPDS[2:0]中设置的 LPD 电压或 V _{IN} 口电压
5	LPDV	LPD 检测电压源 0: 检测电源电压 1: 检测 VLPD 引脚（V _{IN} 口）电压
4	LPDIF	LPD 中断请求标志 0: 无中断挂起读/写 1: 中断挂起
3	LPDMD	LPD 模式选择控制位 0: 当 V _{DD} 电压小于设定的 LPD 检测电压时或 V _{IN} 口低于 1.20V 时，LPDIF 标志置 1

LPDSEL



SH79F326 例程说明

位编号	位符号	说明
3-0	LPDS[3:0]	LPD 电压设置位 0000: 2.4V 0001: 2.55V 0010: 2.7V 0011: 2.85V 0100: 3.00V 0101: 3.15V 0110: 3.30V 0111: 3.45V 1000: 3.60V 1001: 3.75V 1010: 3.90V 1011: 4.05V 1100: 4.20V 1101: 4.35V



2.13 CRC

为提高系统可靠性, SH79F326 内建 1 个 CRC 校验模块, 可用来实时生成 code 的 CRC 校验码, 采用的生成多项式为: $X^{16}+X^{12}+X^5+1$ 。用户可利用此校验码和理论值比较, 监测 Flash 内容是否有变化。通过 CRCMODE 位可以选择校验范围是否包含最后两个 Byte, 或其它位置 (比如类 EEPROM 区, 序列号区, 用户识别码区等)

CRC 校验过程中的读 Flash ROM 动作穿插在 CPU 执行取指的读动作的间隙完成, 不会影响 CPU 的指令执行时间, 但如果 CPU 进入 IDLE 状态, 则 CRC 校验过程会大大加快。

2.13.1 测试程序介绍

Demo 程序中实现了连续 CRC 校验, 一次 CRC 完成后, 会置起中断。中断程序中会再次开启 CRC 校验。

2.13.2 寄存器介绍

CRCCON

位编号	位符号	说明
7	CRC_GO	CRC 启动控制位 0: 关闭 CRC 模块 1: 启动 CRC 模块,CRC 校验完成后自动清 0.
6	CRCIF	CRC 完成中断请求标志位 0: 未启动或启动后未完成。 1: CRC 校验完成, 向 CPU 申请中断, 如果中断允许位 ECRC 为 1, 则 CPU 响应中断, 程序响应中断后此标志位被软件清 0。
5	CRCMODE	CRC 校验模式控制位 0: 校验地址范围最后 2Byte 不计入 CRC 校验 1: 校验地址范围最后 2Byte 计入 CRC 校验
4-0	CRCADDR	CRC 校验范围设置位 00000: 校验地址范围为 0000~07FDH(2K-2Byte) 00001: 校验地址范围为 0000~0FFDH(4K-2Byte) 00010: 校验地址范围为 0000~17FDH(6K-2Byte) 00011: 校验地址范围为 0000~1FFDH(8K-2Byte)



3. 更改记录

版本	记录	日期
1.0	初始版本	2017 年 11 月
2.2	修改笔误	2023 年 8 月



4. 目录

1. 概述.....	1
1.1 整体框架介绍	1
1.2 使用实例说明	2
2. 模块介绍.....	6
2.1. 系统时钟.....	6
2.1.1 工作方式配置介绍.....	6
2.1.2 测试程序介绍.....	6
2.1.3 寄存器简要介绍.....	6
2.2 I/O 端口	8
2.3 定时器/计数器.....	9
2.3.1 工作方式配置介绍.....	9
2.3.2 测试程序介绍.....	9
2.3.3 相关寄存器介绍.....	9
2.4 可编程计数器阵列 PCA0	11
2.4.1 工作方式配置介绍.....	11
2.4.2 测试程序介绍.....	11
2.5 LCD 驱动器	15
2.5.1 寄存器简要介绍.....	15
2.6 LED 驱动器	18
2.6.1 寄存器简要介绍.....	18
2.7 TWI 串行通讯接口	19
2.7.1 工作方式配置介绍.....	19
2.7.2 测试程序介绍.....	19
2.8 UART 接口	19
2.8.1 工作方式配置介绍.....	20
2.8.2 测试程序介绍.....	20
2.8.3 寄存器介绍.....	20
2.9 ADC.....	22
2.9.1 测试程序介绍.....	22
2.9.2 寄存器简要介绍.....	22
2.10 PWM	23
2.10.1 测试程序介绍.....	23
2.10.2 寄存器介绍.....	23
2.11 EEPROM 操作	25
2.12 LPD	25
2.12.1 测试程序介绍.....	25
2.12.2 寄存器介绍.....	25
2.13 CRC.....	27



SH79F326 例程说明

2.13.1	测试程序介绍	27
2.13.2	寄存器介绍	27
3.	更改记录	28
4.	目录	29

